

Junior Backend Developer

ASP.NET Core & SQL Server

The Complete Interview Book

Questions, Answers & Practical Tasks

Everything You Need to Pass Your Backend Interview

100 Questions | 54 Detailed Answers | 10 Practical Tasks | Success Checklist

April 2026

Table of Contents

Part 1: 100 General ASP.NET Questions	3
1. ASP.NET Fundamentals	3
2. ASP.NET Core	5
3. MVC Architecture & Routing	7
4. Entity Framework	9
5. Web API & REST	11
6. Security & Authentication	13
7. Advanced Topics & Best Practices	15
Part 2: Junior Backend Q&A with Detailed Answers	17
Q1-Q10. C# Programming Basics	17
Q11-Q18. ASP.NET Core Fundamentals	21
Q19-Q25. REST API Development	24
Q26-Q36. SQL Server Fundamentals	27
Q37-Q42. Entity Framework Core	31
Q43-Q47. Authentication & Security	33
Q48-Q54. Practical Scenarios	35
Part 3: Practical Interview Tasks	39
Task 1: Build a CRUD REST API	39
Task 2: Write SQL Queries	40
Task 3: Implement JWT Authentication	40
Task 4: Filtering, Sorting & Pagination	41
Task 5: Database Design & ERD	41
Task 6: Global Exception Handling	42
Task 7: Many-to-Many Relationship	42
Task 8: Write a Stored Procedure	43
Task 9: Repository Pattern & Unit of Work	43
Task 10: File Upload & Download	44
Interview Success Checklist	45
Before the Interview	45
During the Interview	45
Common Mistakes That Cost Junior Developers Points	46

Note: Right-click the Table of Contents and select "Update Field" to refresh page numbers.

Part 1: 100 General ASP.NET Questions

This section contains 100 essential ASP.NET interview questions covering all major topics. These questions test your breadth of knowledge across the entire ASP.NET ecosystem, from fundamentals to advanced concepts. Use these to assess your overall readiness and identify areas that need deeper study.

The questions are organized into 7 categories, each with 15 questions. For detailed answers to the most critical questions, refer to Part 2 which provides comprehensive answers specifically targeted at the junior backend level.

1. ASP.NET Fundamentals

1. What is ASP.NET and how does it differ from classic ASP?
2. Explain the ASP.NET page lifecycle. What are the key events in order?
3. What is the difference between `Server.Transfer` and `Response.Redirect`?
4. What are the different types of state management techniques in ASP.NET?
5. Explain the difference between `ViewState`, `SessionState`, and `ApplicationState`.
6. What is the `Global.asax` file and what events does it handle?
7. What is the `web.config` file and what are its key sections?
8. What is the `App_Code` and `App_Data` folder used for?
9. Explain the difference between Web Forms and ASP.NET MVC.
10. What are HTTP Handlers and HTTP Modules? How do they differ?
11. What is the ASP.NET pipeline and how does a request flow through it?
12. What is the difference between in-process and out-of-process session state?
13. Explain the concept of Server Controls in Web Forms.
14. What is `PostBack` in ASP.NET Web Forms and how does it work?
15. What are Master Pages and how do they enable consistent layouts?

16. ASP.NET Core

16. What is ASP.NET Core and what are its key advantages over the traditional ASP.NET Framework?
17. Explain the difference between ASP.NET Core and the .NET Framework.
18. What is the `Startup` class in ASP.NET Core and what are its key methods?
19. What is the purpose of the `Program.cs` file in ASP.NET Core 6 and later?
20. Explain dependency injection in ASP.NET Core. What are the three service lifetimes?
21. What is middleware in ASP.NET Core? How is the request pipeline configured?
22. What is the difference between the development, staging, and production environments?

- 23. What are configuration providers in ASP.NET Core and how is configuration loaded?
- 24. Explain the Kestrel web server. What are its advantages?
- 25. What is the WebHostBuilder and how does it configure the application?
- 26. What are the key differences between ASP.NET Core 3.1, 5.0, 6.0, 7.0, and 8.0?
- 27. What is the appsettings.json file and how does configuration binding work?
- 28. What are tag helpers in ASP.NET Core and how do they differ from HTML helpers?
- 29. What are Razor Pages and how do they compare to MVC?
- 30. What is the IHost interface and how does the Generic Host work in ASP.NET Core?

31. MVC Architecture & Routing

- 31. Explain the Model-View-Controller (MVC) pattern and its benefits.
- 32. What is the role of the Controller in ASP.NET MVC?
- 33. What are Action Methods and Action Results? List common Action Result types.
- 34. What is the difference between ViewData, ViewBag, and TempData?
- 35. Explain attribute routing versus convention-based routing in ASP.NET MVC.
- 36. What are Action Filters? List the different types and explain their use cases.
- 37. What is the difference between ActionResult and ViewResult?
- 38. What are areas in ASP.NET MVC and why would you use them?
- 39. Explain model binding in ASP.NET MVC. How does it work?
- 40. What are partial views and when would you use them?
- 41. What is the Razor view engine and what are its key syntax features?
- 42. How do you implement validation in ASP.NET MVC? Explain Data Annotations.
- 43. What are strongly-typed views and why are they preferred?
- 44. Explain the concept of Areas in ASP.NET MVC and their routing implications.
- 45. What is the difference between Layout pages and partial views?

46. Entity Framework

- 46. What is Entity Framework and what are its main approaches (Code First, Database First, Model First)?
- 47. Explain the difference between Code First, Database First, and Model First approaches.
- 48. What is a DbContext and what role does it play in Entity Framework?
- 49. What are LINQ to Entities and how do they differ from LINQ to Objects?
- 50. Explain lazy loading, eager loading, and explicit loading in Entity Framework.
- 51. What is the difference between IEnumerable and IQueryable in the context of EF?

- 52. What are migrations in Entity Framework Code First and how do they work?
- 53. What are navigation properties and how do they establish relationships?
- 54. What is the AsNoTracking method and when should you use it?
- 55. How does Entity Framework handle transactions?
- 56. What are complex types in Entity Framework and when would you use them?
- 57. Explain the repository pattern and unit of work pattern in the context of EF.
- 58. What is connection resiliency in Entity Framework?
- 59. What are interceptors in Entity Framework and how are they used?
- 60. What is the difference between Entity Framework Core and Entity Framework 6?

61. Web API & REST

- 61. What is ASP.NET Web API and how does it differ from ASP.NET MVC?
- 62. What are RESTful API design principles?
- 63. What are the main HTTP methods used in Web API and what do they represent?
- 64. How does attribute routing work in Web API?
- 65. What is content negotiation in Web API?
- 66. What are the different return types for Web API action methods?
- 67. How do you implement versioning in ASP.NET Web API?
- 68. What are media formatters in Web API and how do they work?
- 69. How do you handle errors and exceptions in Web API?
- 70. What is Swagger/OpenAPI and how do you add it to an ASP.NET Web API?
- 71. What is CORS and how do you configure it in ASP.NET Web API?
- 72. Explain the difference between IHttpActionResult, IActionResult, and ActionResult<T>.
- 73. What are action filters in Web API and how are they used for logging or authorization?
- 74. How do you implement file upload and download in Web API?
- 75. What is OData and how does it integrate with ASP.NET Web API?

76. Security & Authentication

- 76. What are the different types of authentication available in ASP.NET?
- 77. Explain the difference between authentication and authorization.
- 78. What is ASP.NET Identity and how does it work?
- 79. What is the difference between Forms Authentication and Windows Authentication?
- 80. What is Role-Based Authorization and Claims-Based Authorization?
- 81. How does JWT (JSON Web Token) authentication work in ASP.NET Core?

- 82.** What is OAuth 2.0 and how is it used in ASP.NET applications?
- 83.** What is OpenID Connect and how does it differ from OAuth 2.0?
- 84.** What are anti-forgery tokens and how do they prevent CSRF attacks?
- 85.** What is the Authorize attribute and how is it applied?
- 86.** What are security headers (CORS, CSP, X-Frame-Options) and how do you configure them?
- 87.** How do you securely store connection strings and secrets in ASP.NET?
- 88.** What is the Data Protection API in ASP.NET Core?
- 89.** What is HTTPS enforcement and how do you configure it in ASP.NET Core?
- 90.** What is IdentityServer4 and how is it used for token-based authentication?

91. Advanced Topics & Best Practices

- 91.** What is caching in ASP.NET? Explain output caching, data caching, and distributed caching.
 - 92.** What are the SOLID principles and how do they apply to ASP.NET applications?
 - 93.** What is the difference between synchronous and asynchronous programming (async/await)?
 - 94.** What are background tasks and hosted services in ASP.NET Core?
 - 95.** What is SignalR and how does it enable real-time communication?
 - 96.** What is logging in ASP.NET Core? How do you configure different logging providers?
 - 97.** What are health checks in ASP.NET Core and why are they important?
 - 98.** What is the difference between a singleton, scoped, and transient service lifetime?
 - 99.** What is the IOptions, IOptionsSnapshot, and IOptionsMonitor pattern?
 - 100.** How do you implement rate limiting in ASP.NET Core?
 - 101.** What are the common performance optimization techniques for ASP.NET applications?
 - 102.** What is Blazor and how does it compare to traditional ASP.NET applications?
 - 103.** What are microservices and how does ASP.NET Core fit into a microservices architecture?
 - 104.** What is gRPC and how do you implement it in ASP.NET Core?
 - 105.** What is the Minimal API approach in ASP.NET Core 6+?
-

Part 2: Junior Backend Q&A with Detailed Answers

This section contains the most important questions you will face in a junior backend ASP.NET Core + SQL Server interview, each with a comprehensive, detailed answer. These 54 questions cover exactly what interviewers ask at the junior level, with explanations deep enough to demonstrate real understanding.

Q1-10. C# Programming Basics

Q1: What is the difference between value types and reference types in C#?

Answer: Value types store their data directly on the stack and include primitive types such as int, float, double, bool, char, and struct. When you assign a value type to another variable, a copy of the value is created, meaning changes to one variable do not affect the other. Reference types, on the other hand, store a reference (memory address) to the actual data on the heap. Examples include classes, strings, arrays, and objects. When you assign a reference type to another variable, both variables point to the same memory location, so changes through one variable are visible through the other. Understanding this distinction is critical for a backend developer because it affects how data is passed to methods, how memory is managed, and how unexpected side effects can arise when working with objects in multi-layered applications.

Q2: Explain the four pillars of Object-Oriented Programming (OOP).

Answer: Encapsulation is the practice of bundling data and methods into a single unit (class), while restricting direct access using access modifiers like private or protected. This protects the internal state of an object from unintended modification. Abstraction means hiding complex implementation details and exposing only essential features through abstract classes and interfaces. Inheritance allows a class to inherit fields and methods from a base class, promoting code reuse. Polymorphism allows objects to be treated as instances of their parent class, including compile-time polymorphism (method overloading) and runtime polymorphism (method overriding via virtual and override keywords). These four principles are fundamental to writing clean, maintainable, and scalable backend code.

Q3: What is the difference between abstract classes and interfaces?

Answer: An abstract class can contain both method declarations and fully implemented methods, along with fields, constructors, and properties. A class can inherit from only one abstract class due to single inheritance. An interface can only declare method signatures, properties, events, and indexers (prior to C# 8.0). A class can implement multiple interfaces, making them ideal for defining contracts. Use abstract classes for shared code among closely related classes, and interfaces for defining contracts that any class can fulfill, which is especially useful for dependency injection and unit testing.

Q4: What are access modifiers in C#? List and explain each one.

Answer: Public means accessible from any code. Private restricts access to the containing class only. Protected allows access within the containing class and derived classes. Internal restricts access to code within the same assembly. Protected Internal combines both protected and internal access. In backend applications, public is used for service interfaces and controller actions, private for internal helpers, and internal for infrastructure implementations not exposed to other projects.

Q5: What is the difference between string and StringBuilder in C#?

Answer: Strings are immutable in C#: once created, their value cannot change. Concatenating strings with + creates a new object each time, causing performance overhead with many concatenations. StringBuilder is a mutable object with an internal buffer for efficient string manipulation without creating new objects. Use string for simple concatenations (fewer than 5-

10), and StringBuilder in loops, large text processing, or generating dynamic SQL and CSV exports.

Q6: What are generics in C# and why are they useful?

Answer: Generics allow you to define classes, interfaces, and methods with placeholder types (type parameters), typically denoted by T. Instead of writing multiple versions for different data types, you write a single generic class like List<T> that works with any type. This provides compile-time type safety, eliminates boxing/unboxing for value types, and improves code reuse. Generics are used extensively in backend development: repositories, API response wrappers like ApiResponse<T>, and DbSet<T> in Entity Framework.

Q7: What is LINQ? Explain the difference between LINQ to Objects and LINQ to Entities.

Answer: LINQ (Language Integrated Query) allows writing queries directly in C# using method syntax or query syntax. LINQ to Objects operates on in-memory IEnumerable<T> collections; all filtering happens in memory. LINQ to Entities (with Entity Framework) operates on IQueryable<T> and translates expressions into SQL queries executed on the database server. The key difference is execution location: LINQ to Objects runs in application memory, while LINQ to Entities translates to SQL and runs on the database, fetching only necessary data.

Q8: What is the difference between async and await in C#?

Answer: The async keyword marks a method as containing asynchronous operations and transforms it to return a Task or Task<T>. The await keyword pauses method execution until the awaited task completes, without blocking the calling thread. The thread is released back to the thread pool, improving scalability. This is essential in ASP.NET Core for database calls (ToListAsync) and external API calls, preventing thread starvation and improving application throughput.

Q9: What are exceptions in C# and how do you handle them?

Answer: Exceptions are runtime errors that disrupt normal execution. Common types include NullReferenceException, InvalidOperationException, ArgumentException, and SqlException. Handle them using try-catch-finally blocks: try for the risky code, catch for specific exception types, and finally for cleanup. Best practices: catch specific exceptions, avoid empty catch blocks, use the 'using' statement for IDisposable objects, and in ASP.NET Core use global exception handling middleware.

Q10: What is the using statement and IDisposable interface?

Answer: IDisposable defines a Dispose() method for releasing unmanaged resources (database connections, file handles). The 'using' statement ensures Dispose() is called automatically, even if exceptions occur. C# 8.0+ also supports 'using declarations' without braces. In backend development with SQL Server, proper disposal prevents connection leaks, memory leaks, and file lock issues under load.

Q11-18. ASP.NET Core Fundamentals

Q11: What is ASP.NET Core and why choose it over the .NET Framework?

Answer: ASP.NET Core is a modern, cross-platform, high-performance web framework. Unlike the .NET Framework (Windows-only with IIS), it runs on Windows, Linux, and macOS. It uses the lightweight Kestrel server, has built-in dependency injection, unified configuration, and modular packages. Benchmarks consistently show it among the fastest frameworks. It offers better tooling, container-friendly deployment, and access to the latest .NET features.

Q12: Explain Dependency Injection (DI). What are the three service lifetimes?

Answer: DI is a pattern where objects receive dependencies from an external container. The three lifetimes are: Transient (new instance per request), Scoped (one instance per HTTP request, ideal for DbContext), and Singleton (one instance for the application lifetime, for caching and configuration). A critical mistake is injecting a scoped service into a singleton, causing disposed object issues.

Q13: What is middleware and how does the request pipeline work?

Answer: Middleware are components assembled into a pipeline to handle requests and responses. Each receives an HttpContext and either passes control to the next component via next() or short-circuits the pipeline. Configured in Program.cs, common middleware include authentication, authorization, CORS, exception handling, and static files. Each piece of middleware can inspect, modify, or short-circuit the request.

Q14: What is the difference between appsettings.json and environment variables?

Answer: appsettings.json is the primary configuration file with environment-specific overrides (appsettings.Production.json). Environment variables override JSON values and are essential for production/container deployments. Sources load in order: appsettings.json, environment-specific file, environment variables, command-line arguments. Store defaults in JSON; use environment variables or secret managers for sensitive values like connection strings.

Q15: What are Controllers and how do they work?

Answer: Controllers are classes deriving from ControllerBase (APIs) that handle HTTP requests via action methods. The framework maps request data to parameters through model binding. Controllers return various results: OkObjectResult (200), NotFoundResult (404), CreatedAtActionResult (201). The [ApiController] attribute enables automatic 400 responses for validation errors. In a well-structured backend, controllers are thin and delegate to services.

Q16: What is model binding and model validation?

Answer: Model binding maps HTTP request data (route values, query strings, request body) to action parameters or DTOs. Model validation checks data against rules using Data Annotations ([Required], [StringLength], [Range], [EmailAddress]). With [ApiController], invalid models automatically return 400 with error details. Always validate server-side; never trust client-side validation alone.

Q17: Explain the Program.cs file and application startup.

Answer: In ASP.NET Core 6+, Program.cs uses top-level statements: WebApplication.CreateBuilder() initializes the builder; builder.Services.Add...() registers DI

services; `builder.Build()` creates the app; `app.Use...()` configures middleware; `app.Run()` starts listening. Every feature (database, auth, caching, middleware) is registered and configured here.

Q18: What is the difference between development and production mode?

Answer: The `ASPNETCORE_ENVIRONMENT` variable controls the environment. Development enables detailed error pages (`UseDeveloperExceptionPage`), Swagger UI, hot reload, and verbose logging. Production disables these for security, using `UseExceptionHandler` for user-friendly errors and Information/Warning-level logging. Use `IWebHostEnvironment.IsDevelopment()` to conditionally configure features.

Q19-25. REST API Development

Q19: What are RESTful API design principles?

Answer: REST principles include: statelessness (each request is independent), resource-based URLs (nouns, not verbs like `/api/customers`), standard HTTP methods (GET for read, POST for create, PUT for replace, PATCH for partial update, DELETE for remove), proper status codes (200, 201, 204, 400, 401, 403, 404, 500), and consistent response formats. Following these ensures predictable, consumable APIs.

Q20: What are HTTP status codes and when to use them?

Answer: 200 OK for successful retrieval/update. 201 Created for new resource creation. 204 No Content for successful operations with no body (DELETE). 400 Bad Request for invalid input. 401 Unauthorized for missing credentials. 403 Forbidden for insufficient permissions. 404 Not Found for missing resources. 409 Conflict for state conflicts. 500 Internal Server Error for unexpected failures. Correct codes help frontend apps handle responses properly.

Q21: What is the difference between PUT, PATCH, and POST?

Answer: POST creates new resources (not idempotent; returns 201 Created). PUT replaces an entire resource (idempotent; requires full data). PATCH partially updates a resource (idempotent; sends only changed fields). PUT is simpler but less efficient for partial updates; PATCH is more bandwidth-efficient but requires careful merge logic on the server.

Q22: What is Swagger/OpenAPI and why is it important?

Answer: Swagger generates interactive API documentation showing all endpoints, methods, parameters, and schemas. Added via `Swashbuckle.AspNetCore`, it auto-generates from controller code and XML comments. It serves as a contract between frontend/backend teams, enables testing without Postman, and provides onboarding documentation for new developers. Always include XML comments on controllers and DTOs for meaningful descriptions.

Q23: What is CORS and how do you configure it?

Answer: CORS is a browser security feature restricting cross-origin requests. Servers include `Access-Control-Allow-Origin` headers to permit specific origins. Configure in `Program.cs`: `builder.Services.AddCors()` defines policies, `app.UseCors()` applies them. Use `AllowAnyOrigin()` in development only; in production, specify exact allowed origins for security. CORS is browser-only and does not protect against server-to-server calls.

Q24: How do you implement pagination in a REST API?

Answer: Accept pageNumber and pageSize query parameters. Use LINQ Skip((page-1)*size).Take(size) on IQueryable to execute pagination at the database level. Return a PagedResult<T> with data, TotalCount, TotalPages, CurrentPage, and HasNextPage. Always ensure pagination happens in the database query, not in memory after fetching all records.

Q25: What is a DTO and why not return entities directly?

Answer: A DTO (Data Transfer Object) carries data between layers, separate from database entities. Returning entities directly is bad because it can expose sensitive data (passwords, internal IDs), tightly couples API to database schema, may cause serialization issues with lazy-loaded navigation properties, and prevents different shapes for different operations. Use separate DTOs for create, update, and response scenarios.

Q26-36. SQL Server Fundamentals**Q26: What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?**

Answer: INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table with matching right-table data (NULLs if no match). RIGHT JOIN returns all rows from the right table with matching left-table data. FULL OUTER JOIN returns all rows from both tables, matching where possible with NULLs for unmatched rows. LEFT JOIN is most common for optional relationships; INNER JOIN for required ones.

Q27: What is the difference between WHERE and HAVING?

Answer: WHERE filters individual rows before GROUP BY and cannot use aggregate functions. HAVING filters groups after GROUP BY and can reference aggregates like COUNT and SUM. Execution order: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. Use WHERE for row-level filtering (improves performance by reducing rows early) and HAVING for group-level filtering.

Q28: What is the difference between UNION and UNION ALL?

Answer: UNION combines result sets and removes duplicates (requires sorting, slower). UNION ALL combines without removing duplicates (faster). Both require same column count and compatible types. Use UNION ALL when duplicates are acceptable or impossible, as it can be orders of magnitude faster by avoiding the sort/dedup step.

Q29: What is a stored procedure?

Answer: A stored procedure is a precompiled, reusable set of SQL statements stored on the server. Advantages: precompiled execution plans (faster), security (grant execute permission without table access), reduced network traffic (single EXECUTE call), code reusability, and centralized business logic. In modern ASP.NET Core, use EF LINQ for simple CRUD and stored procedures for complex, performance-critical operations.

Q30: What is the difference between stored procedures and functions?

Answer: Stored procedures can perform DML operations (INSERT, UPDATE, DELETE), return multiple result sets, output parameters, and return codes. Functions must return a single value (scalar) or table, cannot modify data, and are called within SQL statements. Use procedures for data-modifying business logic; use functions in SELECT statements for computed columns.

Q31: What are indexes and how do they improve performance?

Answer: Indexes improve data retrieval speed at the cost of additional storage and slower writes. A clustered index determines physical data order (one per table, typically the primary key). Non-clustered indexes create separate structures pointing to data rows (up to 999 per table). Add indexes to columns in WHERE, JOIN, and ORDER BY clauses. Balance query performance gains against write operation overhead.

Q32: What is the difference between DELETE, TRUNCATE, and DROP?

Answer: DELETE is DML: removes rows one by one, fully logged, can have WHERE clause, fires triggers, and can be rolled back. TRUNCATE is DDL: deallocates data pages (much faster), removes ALL rows, resets identity, no triggers, cannot be used with foreign keys. DROP is DDL: removes the entire table structure including data, indexes, and constraints. Use DELETE for selective removal, TRUNCATE for clearing all rows, DROP only to remove the table entirely.

Q33: What is a transaction and what are ACID properties?

Answer: A transaction groups multiple SQL operations into one atomic unit. ACID: Atomicity (all succeed or all rollback), Consistency (valid state to valid state), Isolation (concurrent transactions do not interfere), Durability (committed changes survive failures). In SQL Server: BEGIN TRAN, COMMIT, ROLLBACK. In EF Core, SaveChanges() wraps operations in a transaction automatically. Essential for multi-table operations like order processing.

Q34: What is the difference between VARCHAR and NVARCHAR?

Answer: VARCHAR stores non-Unicode data using 1 byte per character (up to 8000). NVARCHAR stores Unicode data using 2 bytes per character (up to 4000), supporting all languages and special characters. Use VARCHAR for ASCII-only data (emails, URLs, system codes) to save space. Use NVARCHAR as default for multilingual or user-generated content. Both have MAX variants for up to 2GB.

Q35: What are CTEs and how are they used?

Answer: Common Table Expressions are temporary named result sets defined with WITH. Non-recursive CTEs are simple subqueries for readability. Recursive CTEs have an anchor member and a self-referencing member connected by UNION ALL, used for hierarchical data (org charts, category trees). CTEs exist only within the statement scope and improve readability over nested subqueries.

Q36: What are the different types of constraints?

Answer: Primary Key: uniquely identifies each row, no NULLs. Foreign Key: establishes relationships, ensures referential integrity. Unique: distinct values in a column, allows one NULL. Check: enforces a condition (price > 0). Not Null: column cannot contain NULL. Default: provides

a default value on INSERT. Define constraints at the database level; EF Core maps them through attributes or Fluent API.

Q37-42. Entity Framework Core

Q37: What is Entity Framework Core?

Answer: EF Core is Microsoft's ORM for .NET, allowing you to work with database data using C# objects. It maps entity classes to tables, translates LINQ to SQL, and manages change tracking. The DbContext is the primary class for database interaction. DbSet<T> properties represent tables. It supports SQL Server, PostgreSQL, MySQL, and SQLite through provider packages. Migrations provide version-controlled schema evolution.

Q38: What is the difference between Code First, Database First, and Model First?

Answer: Code First: define C# classes first, generate database via migrations. Database First: reverse-engineer classes from existing database using Scaffold-DbContext. Model First: design visually, generate both database and classes. Code First with Migrations is recommended for new projects (version control, CI/CD friendly). Database First is for legacy databases.

Q39: Explain lazy loading, eager loading, and explicit loading.

Answer: Lazy loading: automatically loads navigation properties on first access (requires virtual properties, can cause N+1 problem). Eager loading: loads related data upfront using Include() (recommended, single query). Explicit loading: manually loads via Entry().Collection().Load(). Prefer eager loading for known data needs; avoid lazy loading in APIs due to N+1 performance issues and serialization problems.

Q40: What are migrations?

Answer: Migrations incrementally update the database schema to match entity classes. Add-Migration generates Up() (apply changes) and Down() (reverse changes) methods. Update-Database applies them. Migration files are C# code checked into source control. They provide version-controlled, auditable schema evolution. In production, apply via dotnet ef database update or programmatically at startup.

Q41: What is the difference between IEnumerable and IQueryable?

Answer: IEnumerable<T> operates in-memory; LINQ methods execute by the .NET runtime. IQueryable<T> represents a query stored as an expression tree translated to SQL by the database provider. Only terminal operations (ToList, FirstOrDefault) trigger execution. Converting to IEnumerable early (AsEnumerable/ToList) causes in-memory processing. Keep queries as IQueryable as long as possible for optimal database-side execution.

Q42: What is tracking and AsNoTracking()?

Answer: By default, EF Core tracks entity changes for SaveChanges(). Tracking creates snapshots and maintains change proxies, adding memory and performance overhead. AsNoTracking() disables tracking for read-only queries, reducing memory and improving performance. Use it for listing, displaying data, and dropdown populations. Use tracking (default) when you need to modify and save entities.

Q43-47. Authentication & Security

Q43: What is the difference between authentication and authorization?

Answer: Authentication verifies identity (Who are you?) using credentials, tokens, or certificates. Authorization determines permissions (What can you do?) based on roles, claims, or policies. Authentication is configured with `AddAuthentication()`; authorization with `AddAuthorization()`. `[Authorize]` attribute enforces both. They address different security concerns and work together sequentially.

Q44: What is JWT and how does it work?

Answer: JWT (JSON Web Token) is an open standard for transmitting information as a JSON object. It has three parts: Header (algorithm), Payload (claims like `userId`, `role`, `expiration`), and Signature (ensures integrity). In ASP.NET Core: user sends credentials to `/login`, server validates and returns JWT. Client sends token in Authorization header as `Bearer {token}`. Server validates signature and extracts claims. JWTs are stateless, ideal for distributed applications.

Q45: How do you securely store passwords?

Answer: Never store plain text passwords. Use ASP.NET Core Identity's `PasswordHasher<T>`, which uses PBKDF2 with HMAC-SHA256, random salt, and thousands of iterations. Each registration generates a unique salt combined with the password and hashed iteratively. On login, the same process repeats with the stored salt. Never use MD5/SHA-1 (too fast), never encrypt (reversible), and never write custom hashing algorithms.

Q46: What are SQL injection attacks and how to prevent them?

Answer: SQL injection inserts malicious SQL through user input. For example, `' OR 1=1 --` in a login field bypasses authentication. Primary defense: use parameterized queries, which EF Core does automatically by default. For raw SQL, use `FromSqlInterpolated`. Additional defenses: input validation, stored procedures with parameters, least-privilege database accounts, and never concatenating user input into SQL strings.

Q47: What is CORS, CSRF, and XSS?

Answer: CORS controls which domains can access your API (misconfiguration allows cross-origin attacks). CSRF tricks a browser into submitting authenticated requests to your app (defend with anti-forgery tokens and SameSite cookies). XSS injects malicious scripts into pages (defend with output encoding, CSP headers, and input validation). In backend APIs returning JSON, XSS is less of a concern, but CSRF and CORS are critical.

Q48-54. Practical Scenarios

Q48: How would you design an API with filtering, sorting, and pagination?

Answer: Create a query DTO with `SearchTerm`, `CategoryId`, `MinPrice`, `MaxPrice`, `SortBy`, `SortDirection`, `PageNumber`, and `PageSize`. Build an `IQueryable`, apply filters conditionally, count total before pagination, apply dynamic sorting with `OrderBy/OrderByDescending`, then `Skip/Take`. Return `PagedResult<T>` with data and metadata. All operations execute at database level via `IQueryable` for optimal performance.

Q49: How would you handle errors globally?

Answer: Create custom exception middleware with try-catch around `next()`. Map custom exceptions (`NotFoundException` to 404, `ValidationException` to 400) to correct status codes. Log exceptions with `ILogger`. Return consistent JSON error response (`ProblemDetails`). For unhandled exceptions, return generic 500 without stack traces. Configure in `Program.cs` before routing middleware.

Q50: How would you implement logging?

Answer: Use `ILogger<T>` injected via DI. Log levels: Trace (detailed debug), Debug (diagnostic), Information (operational events like user login), Warning (abnormal events), Error (exceptions), Critical (failures). Use Serilog for production: structured logging, multiple sinks (console, file, database), context enrichment. Best practices: use structured logging, log at appropriate levels, never log sensitive data, and avoid logging inside tight loops.

Q51: How would you connect to SQL Server?

Answer: Install `Microsoft.EntityFrameworkCore.SqlServer`. Create entity classes and `DbContext` with `DbSet<T>` properties. Configure connection string in `appsettings.json` under `ConnectionStrings`. Register with `builder.Services.AddDbContext<AppDbContext>(opt => opt.UseSqlServer(...))`. Create initial migration with `Add-Migration` and `Update-Database`. Use Windows Auth (`Trusted_Connection`) or SQL Auth. Store connection strings in environment variables for production.

Q52: What is the N+1 query problem?

Answer: Loading N entities then accessing each one's navigation property in a loop generates N+1 queries (1 for entities + N for related data). Solution: eager loading with `Include()` generates a single JOIN query. Use `ThenInclude()` for multiple levels. Alternative: `Select()` to project specific data into DTOs. Detect by enabling EF Core SQL logging or using profiling tools like `MiniProfiler`. Always prefer eager loading for known data needs.

Q53: What would a coding exercise for this role look like?

Answer: Build a Task Management API: CRUD endpoints for tasks (Title, Description, Status, Priority, CreatedDate, DueDate). GET `/api/tasks` with status filter and pagination, GET `/api/tasks/{id}`, POST `/api/tasks`, PUT `/api/tasks/{id}`, DELETE `/api/tasks/{id}`. Evaluators look for: proper project structure, DTOs, validation, async operations, DI, correct HTTP status codes, error handling, pagination, and clean code with proper naming.

Q54: How would you deploy an ASP.NET Core application?

Answer: Options: `dotnet publish -c Release` produces self-contained or framework-dependent output. Host on IIS with ASP.NET Core Module (Windows). Docker with Kestrel base image for containerized deployment. Azure App Service for managed hosting. Always: configure Production environment, set connection strings via environment variables, enable HTTPS, configure logging, and add health check endpoints.

Part 3: Practical Interview Tasks

This section covers the 10 most common hands-on coding tasks you will encounter in interviews. Each task includes a description, what the evaluator looks for, and key implementation points. These tasks test your ability to write real code, not just answer theoretical questions.

Task 1: Build a CRUD REST API

The single most common interview task. Create a complete Product API with GET (list with pagination), GET by ID, POST, PUT, DELETE (soft delete). Use DTOs, validation with Data Annotations, async operations, and proper HTTP status codes (200, 201, 204, 400, 404).

What the Evaluator Looks For:

- Project structure: Controllers, Services, Repositories, Data layers
- DTOs instead of entity exposure
- ToListAsync, FirstOrDefaultAsync, SaveChangesAsync
- DI registration for DbContext and services
- Pagination with Skip and Take on IQueryable
- [Required], [MaxLength], [Range] validation attributes
- NotFound checks returning 404
- Clean naming conventions

Task 2: Write SQL Queries

Most interviews include a written SQL assessment. Given Orders and OrderItems tables, write queries for: orders in last 30 days over \$500, top 5 customers by spending, products never ordered, revenue per month, customers with 3+ orders last month, duplicate orders, and update stock in a transaction.

What the Evaluator Looks For:

- Correct JOINS (INNER, LEFT)
- WHERE with date comparisons (GETDATE, DATEDIFF)
- GROUP BY with SUM, COUNT, AVG
- HAVING for group filtering
- ORDER BY with TOP or OFFSET-FETCH
- Subqueries and CTEs (WITH clause)
- Transactions: BEGIN TRAN, COMMIT, ROLLBACK
- Parameterized thinking with @variables

Task 3: Implement JWT Authentication

Build a login/register system: POST /api/auth/register (hash password with PasswordHasher), POST /api/auth/login (validate and return JWT), [Authorize] for protected endpoints, [Authorize(Roles="Admin")] for admin-only endpoints. Configure JwtBearer in Program.cs.

What the Evaluator Looks For:

- PasswordHasher for password hashing (never plain text)
- JWT generation with claims (UserId, Role)
- AddAuthentication().AddJwtBearer() configuration
- [Authorize] and role-based authorization
- 401 for unauthenticated, 403 for unauthorized
- Secrets from configuration, not hardcoded

Task 4: Filtering, Sorting & Pagination

Build GET /api/employees with dynamic filtering (department, salary range, search term), sorting by any field with asc/desc, and pagination. Return PagedResult with TotalCount, Page, TotalPages, HasNext. All operations execute at database level on IQueryable.

What the Evaluator Looks For:

- Build query as IQueryable, execute with ToListAsync at end
- Conditional WHERE clauses (only when params provided)
- Dynamic sorting with Expression<Func<T, object>>
- Calculate total count before Skip/Take
- PagedResult wrapper with metadata
- Never convert to IEnumerable/List too early

Task 5: Database Design & ERD

Design tables for e-commerce: Customers (Id, Name, Email, Phone), Products (Id, Name, Price, Stock), Orders (Id, CustomerId FK, Status, TotalAmount), OrderItems (Id, OrderId FK, ProductId FK, Quantity, UnitPrice). Write CREATE TABLE with PKs, FKs, constraints.

What the Evaluator Looks For:

- Correct data types (INT, DECIMAL, NVARCHAR, DATETIME)
- Primary keys on all tables (IDENTITY)
- Foreign key constraints with CASCADE rules
- UNIQUE on Email, CHECK on Price > 0
- One-to-many and many-to-many relationships
- Normalized design (no duplication)
- EF Core navigation properties

Task 6: Global Exception Handling

Create custom exception middleware catching all unhandled exceptions. Map NotFoundException to 404, ValidationException to 400. Log exceptions, return consistent JSON error response. Generic 500 for unexpected errors without stack trace exposure.

What the Evaluator Looks For:

- try-catch around next() invocation
- Consistent error response format
- Status code mapping per exception type
- Logging before response
- No stack traces to clients
- Middleware pipeline order

Task 7: Many-to-Many Relationship

Implement Students-Courses with Enrollment join entity (StudentId, CourseId, EnrolledDate, Grade). Configure composite key. Endpoints: enroll student in course, list courses for student, list students in course using Include() for eager loading.

What the Evaluator Looks For:

- Join/bridge table for many-to-many
- Foreign keys on join entity
- Navigation properties on all entities
- Composite primary key configuration
- Include() for eager loading
- Duplicate enrollment prevention

Task 8: Write a Stored Procedure

Create a stored procedure for monthly sales report accepting @StartDate and @EndDate. Return TotalOrders, TotalRevenue, AverageOrderValue, TopSellingProduct, NewCustomers. Call from EF Core using FromSqlRaw or ExecuteSqlRaw.

What the Evaluator Looks For:

- CREATE PROCEDURE syntax with parameters
- Aggregations: SUM, COUNT, AVG
- Output parameters or result sets
- FromSqlRaw in EF Core
- BEGIN TRY / BEGIN CATCH for error handling

Task 9: Repository Pattern & Unit of Work

Create generic IRepository with GetAllAsync, GetByIdAsync, FindAsync, Add, Update, Delete. Implement Repository and IUnitOfWork coordinating multiple repositories with SaveAsync. Register with Scoped lifetime in DI.

What the Evaluator Looks For:

- Generic repository with DbSet and async
- IQueryable return types for composition
- Unit of Work with shared context

- Scoped DI registration
- Clean architecture and testability

Task 10: File Upload & Download

POST /api/files/upload accepting IFormFile. Validate image extensions (jpg, png, gif) and size (5MB max). Save with GUID filename. GET /api/files/{filename} returning PhysicalFileResult. Prevent path traversal attacks.

What the Evaluator Looks For:

- IFormFile for uploads
 - Extension and size validation
 - GUID-based unique filenames
 - Path.Combine for safe paths
 - PhysicalFileResult for downloads
 - Path.GetFileName to prevent traversal
-

Interview Success Checklist

Before the Interview

- Set up a project template with Controllers/, Services/, Data/, Models/, DTOs/ folders
- Have SQL Server ready (localdb or Docker) with sample databases for practice
- Memorize: async methods (ToListAsync, SaveChangesAsync), DI registration, middleware order
- Practice CRUD API from scratch at least 3-5 times until it is muscle memory
- Practice writing SQL JOINS, GROUP BY + HAVING, and subqueries by hand

During the Interview

- Ask clarifying questions before writing any code
- Build bottom-up: entities, DbContext, DTOs, service, then controller
- Communicate your thought process out loud continuously
- Always handle: NotFound (404), validation (400), and exceptions
- Use async/await for all database operations
- Return DTOs, never database entities, from API controllers
- If stuck, explain what you are trying to achieve - interviewers will guide you

Common Mistakes That Cost Junior Developers Points

- Returning entities instead of DTOs from controllers

- Using synchronous methods (ToList, SaveChanges) instead of async
- Forgetting NotFound handling in GET by ID, PUT, and DELETE
- Not using AsNoTracking() for read-only queries
- Hardcoding connection strings instead of using IConfiguration
- SQL injection by concatenating user input into query strings
- Not disposing resources (missing using statements)